

ExpenseSense: Privacy-Preserving On-Device Financial Tool Calling with Sub-2B LLMs

Anonymous ACL submission

Abstract

Personal finance data—detailed records of when, where, and how a user spends money—is among the most sensitive information individuals possess, making cloud-based LLM inference undesirable for many users. On-device inference addresses this privacy concern by keeping all financial data strictly local, yet the tool-calling capabilities of sub-2B quantized small language models (SLMs) on consumer hardware remain under-studied. We present **ExpenseSense**, a privacy-first personal finance Q&A assistant, and use it as a case study to benchmark on-device tool calling under strict privacy constraints. We design a 115-question test suite spanning five analysis tools across three complexity tiers (L1–L3), defined by a 7-feature difficulty scoring function. We compare a *single-agent* pipeline (joint intent classification and parameter extraction) against a *dual-agent* pipeline (a lightweight router followed by a tool-specific specialist) across six models (0.8B–2B parameters), all running locally via llama.cpp on an Apple M1 (8 GB unified memory). While improved prompt engineering enables the single-agent Qwen3.5-2B to reach 92.0% Task Accuracy, dual-agent decomposition is critical for *strict correctness*: Correct Ratio improves from 60.2% to 77.6% (+17.4 pp) for Qwen3.5-2B, and Task Accuracy improves from 79.8% to 89.1% (+9.3 pp) for the smaller Qwen3.5-0.8B. Error taxonomy analysis reveals a qualitative shift: single-agent errors are dominated by `PARAM_EXTRA` and `PARAM_MISSING`, while dual-agent residual errors consolidate into `PARAM_MISSING` on complex queries—a more predictable and safer failure mode. The Qwen3.5-0.8B dual configuration achieves 89.1% Task Accuracy at 3.0 s latency and ≈ 1.3 GB RAM, offering a practical deployment point for privacy-preserving financial assistants where no user data ever leaves the device. All code, prompts, benchmark data, and results are publicly released.

1 Introduction

Large language models (LLMs) have rapidly advanced the state of tool-augmented AI (Schick et al., 2023; Patil et al., 2024; Qin et al., 2024), enabling conversational agents to invoke structured APIs from natural language. Yet almost all tool-calling research targets cloud-hosted models with tens or hundreds of billions of parameters. For personal financial applications, cloud inference raises concerns along three axes: latency, cost, and—most critically—*privacy*, since expense records reveal detailed behavioural patterns that many users are unwilling to transmit to third-party servers.

Personal finance is among the strongest motivating domains for human-centered privacy in AI. Expense tracking databases contain fine-grained records of when, where, and how a user spends money—data that constitutes what Nissenbaum (2004) terms a violation of *contextual integrity* when it flows outside the user’s expected informational norms. Recent work has shown that LLMs can infer sensitive attributes (location, income, health conditions) from seemingly innocuous text (Staab et al., 2024), making the transmission of financial queries to cloud endpoints a significant privacy risk even when the raw data itself is not uploaded. At the same time, users expect natural language interfaces—asking “how much did I spend on food last month?” or “compare my grocery spending this Jan vs. Jan last year”—that map to structured queries over their local data (Li et al., 2024). This creates an underexplored problem: can *sub-2B quantized models* running entirely on consumer hardware reliably call domain-specific financial tools from informal, colloquial natural language, thereby eliminating the need for cloud inference altogether?

Existing benchmarks such as BFCL (Patil et al., 2025), ToolBench (Qin et al., 2024), and the recent FinMCP-Bench (Zhu et al., 2026) and Fin-

Trace (Cao et al., 2026) either target models above 7B parameters—prohibitively slow on consumer hardware even after quantization—or require cloud inference, entirely bypassing the privacy requirement. CallNavi (Song et al., 2025) is the closest prior work, proposing a two-stage routing architecture for function calling, but evaluates models $\geq 7B$ on general-purpose APIs without the memory and latency constraints of local inference. Recent on-device function-calling models such as Octopus v2 (Chen and Li, 2024) and Hammer (Lin et al., 2024) demonstrate that sub-4B models can achieve competitive tool-calling performance through specialised training, but neither evaluates on domain-specific financial schemas nor reports latency on consumer hardware under strict privacy constraints.

We address three research questions (RQ):

- **RQ1:** Can sub-2B quantized SLMs achieve reliable personal finance analytics tool calling on consumer hardware, enabling fully private local inference?
- **RQ2:** Does decomposing intent classification from parameter extraction (dual-agent) yield significant accuracy and strict correctness gains over a joint single-agent architecture?
- **RQ3:** What are the primary failure modes across architectures, and how do they differ qualitatively?

This paper introduces **ExpenseSense**, with the following contributions:

1. A **115-question benchmark** for financial tool calling over five analytical tools, with a 7-feature complexity scoring function producing three difficulty tiers (L1–L3), designed to simulate real-world human interactions with financial assistants.
2. A comparative evaluation of **single-agent vs. dual-agent** architectures across six quantized SLMs (0.8B–2B parameters), demonstrating that while improved single-agent prompts can reach high Task Accuracy (92.0%), dual-agent decomposition is critical for strict exact-match correctness (+17.4 pp Correct Ratio) and for stabilizing smaller sub-1B models without cloud dependence.

3. A **fine-grained error taxonomy** (7 error types) revealing qualitatively different failure modes between architectures—dual-agent consolidates errors into more predictable categories, a desirable property for safety-critical financial applications.

4. Practical **on-device deployment guidance** for human-centered privacy: Qwen3.5-0.8B in dual mode achieves 89.1% Task Accuracy at 3.0 s latency and ≈ 1.3 GB RAM, demonstrating that users need not sacrifice their financial privacy for high-quality AI analytics.

2 Privacy Motivation

The deployment of LLMs in personal finance represents a paradigmatic case of the tension between AI capability and human privacy. Cloud-based LLM services—whether general-purpose assistants or specialised financial tools—require users to transmit queries containing detailed spending information to external servers. Even when the raw financial database is not uploaded, the *queries themselves* reveal sensitive information: a question like “how much did I spend on medical expenses last month?” discloses both health-related spending and temporal patterns. Staab et al. (2024) demonstrate that LLMs can infer personal attributes from text with high accuracy, and Yao et al. (2024) catalogue the broad landscape of LLM privacy risks including data leakage, memorization, and inference attacks.

ExpenseSense addresses this threat by adopting an *on-device, privacy-first* architecture. By leveraging sub-2B quantized models that run efficiently on consumer hardware (Apple M1 with 8 GB unified memory), all user data and queries remain strictly local. No financial records, no query text, and no model outputs ever leave the device. This design aligns with Nissenbaum’s framework of *contextual integrity*: financial data generated within the context of personal expense tracking should remain within that context, not flow to third-party model providers whose informational norms may differ from the user’s expectations.

The key challenge is that on-device inference with sub-2B models has historically been unreliable for structured tool calling (Kavathekar et al., 2025). Our work demonstrates that architectural decomposition—separating intent classification from parameter extraction—bridges this reliability gap, making on-device private inference a

practical reality rather than a theoretical aspiration.

3 Related Work

Tool-Augmented Language Models. Toolformer (Schick et al., 2023) demonstrated self-supervised tool-use learning; Gorilla (Patil et al., 2024) and ToolLLM (Qin et al., 2024) scaled to thousands of real-world APIs. HuggingGPT (Shen et al., 2023) and ToolAlpaca (Tang et al., 2023) explored multi-step orchestration. APIGen (Liu et al., 2024b) introduced a verified pipeline for generating function-calling training data, showing that even 1B-parameter models fine-tuned on high-quality synthetic data can surpass GPT-3.5-Turbo on structured tool invocation. All of these works target models $\geq 7\text{B}$ with cloud inference, leaving the privacy-constrained on-device setting unexplored.

Function-Calling Evaluation. The Berkeley Function Calling Leaderboard (Patil et al., 2025) established a comprehensive benchmark using AST substring matching and execution-based validation. Kavathekar et al. (2025) proposed an evaluation suite—FSP, ACS, AVC, Task Accuracy, and Correct Ratio—specifically designed for SLM function call generation, operating on JSON dictionaries without code execution; we adopt these metrics directly. CallNavi (Song et al., 2025) proposed a two-stage router+specialist approach—the closest architectural analogue to our dual-agent design—but evaluates models $\geq 7\text{B}$ on general-purpose APIs. We study the orthogonal case of sub-2B models on a domain-specific, single-turn financial schema under strict privacy constraints.

On-Device Function Calling. MobileLLM (Liu et al., 2024a), Phi-3 (Abdin et al., 2024), and related surveys (Xu et al., 2024) have explored sub-4B parameter models for mobile deployment. Octopus v2 (Chen and Li, 2024) fine-tuned a 2B model for on-device API calling via functional tokens, achieving competitive latency against GPT-4 on specific tasks. Hammer (Lin et al., 2024) introduced function masking to reduce overfitting to function names, a problem we observe indirectly through our category normalisation errors. Apple Intelligence (Gunter et al., 2024) validated on-device tool calling at scale with a $\sim 3\text{B}$ model demonstrating that the paradigm is production-ready. Kavathekar et al. (2025) found that structured output compliance is the primary bottleneck

Tool	ID	Key Params
plot_time_series	1	category, year/month, ranges, months, ignore_rent
plot_distribution	2	category, year/month/day, ranges, months, ignore_rent
plot_comparison_bars	3	category, y1/m1, y2/m2, sm/em/ey, ignore_rent
calculate_total	4	category, remarks, year/month/day, months, ignore_rent
get_top_expenses	5	n, category, year/month/day, months, min_amount, ignore_rent

Table 1: Tool inventory. The five tools span visualisation (1–3), aggregation (4), and retrieval (5).

for SLMs—a finding that directly motivates our architectural decomposition approach.

Financial NLP and Privacy. FinQA (Chen et al., 2021) and FinGPT (Yang et al., 2023) target financial numerical reasoning and generation, but do not address tool-calling or privacy constraints. FinMCP-Bench (Zhu et al., 2026) and FinTrace (Cao et al., 2026) evaluate LLM agents for financial tool use, but target cloud-scale models. Personal finance assistants (Li et al., 2024) represent an emerging application domain where on-device inference is especially valuable, as users may prefer to keep financial data local for privacy or lack affordable access to cloud-hosted LLMs.

4 System Design

4.1 Tool Inventory

ExpenseSense exposes five analytical tools over a user’s expense dataframe (columns: date, amount, major category, subcategory, remarks). Table 1 summarises each tool and its parameter space. The tools cover core personal finance use cases: time series analysis, proportional breakdown, period comparison, sum aggregation, and ranked retrieval.

4.2 Architectural Configurations

We evaluate two distinct configurations for mapping natural language queries to tool calls, both designed for fully local, private execution.

Single-Agent Pipeline. One model receives a comprehensive prompt containing all five tool definitions, parameter schemas, and cross-tool few-shot examples. It must jointly determine the correct tool *and* extract all parameters in one inference pass, outputting a JSON object with a `tool` field (1–5) and corresponding parameters. The prompt includes structured output rules, date-handling in-

structions, and 7 tool-specific examples aggregated across all 5 tools. Prompt length: $\approx 5,500$ tokens (context window inclusive).

Dual-Agent Pipeline. The task is decomposed into two sequential inference calls:

1. **Router:** A focused intent-classification prompt with 17 few-shot examples and explicit disambiguation rules (e.g., “compare” requires two distinct time periods) elicits a single digit (1–5).

2. **Specialist:** The predicted tool ID selects a tool-specific prompt containing 12–15 examples for that *one* tool, along with detailed parameter definitions and date-handling rules.

The router sees ≈ 600 tokens; the specialist sees ≈ 800 –1,000 tokens with deeper, more directly relevant supervision. This reduces per-call cognitive load and provides richer in-context learning per subtask, at the cost of two sequential inference passes. Combined prompt tokens: $\approx 2,700$ per query.

4.3 Post-Hoc Parameter Validation

After each inference call, a deterministic `validate_and_fix_params` layer corrects category strings without additional model calls:

1. **Exact match** against the known vocabulary.

2. **Morphological normalisation:** `groceries` $\rightarrow$ `grocery`, `cafés` $\rightarrow$ `cafe`.

3. **Edit-distance matching** via `difflib` (cutoff ≥ 0.80).

4. **Token overlap scoring** (Jaccard; cutoff ≥ 0.50).

All results report both raw (pre-validation) and validated (post-validation) metrics.

5 Benchmark Design

5.1 Test Suite

The benchmark comprises 115 test cases across five task groups (Table 2). Queries are written in naturalistic, informal language: abbreviations (“*compare electricity ’24 vs ’25*”), colloquialisms (“*can ya tell me*”), misspellings (“*distribution*”), and domain-specific terms (“*combinis*”, “*nomikais*”, “*shinkansen*”). This reflects the Japan-based personal finance context of the underlying system

Task Group	Cases	Avg Params	Avg C
Time Series	23	2.9	7.0
Distribution	21	2.5	6.4
Comparison	28	4.4	8.8
Calculate Total	22	2.4	6.3
Top Expenses	21	3.2	6.7
Total	115	3.2	7.1

Table 2: Test case distribution by group. Comparison queries are the most parameter-dense and complex.

and tests robustness to real-world query diversity—the kind of informal, spontaneous interactions that characterise actual use of personal financial assistants.

5.2 Complexity Scoring

Each test case is assigned a difficulty score:

$$C = n_p + d_{\text{date}} + d_{\text{cat}} + d_{\text{rel}} + d_{\text{multi}} + d_{\text{abbr}} + d_{\text{hol}} \quad (1)$$

where n_p = number of expected parameters; $d_{\text{date}} \in \{0-4\}$ scores date specification complexity; $d_{\text{cat}} \in \{0-2\}$ is the category normalisation distance; $d_{\text{rel}} \in \{0, 1\}$ flags relative time expressions; $d_{\text{multi}} \in \{0-2\}$ counts compound parameter groups; $d_{\text{abbr}} \in \{0, 1\}$ flags abbreviated dates; and $d_{\text{hol}} \in \{0, 2\}$ flags calendar knowledge requirements. Scores are binned into three tiers: **L1** ($C \leq 4$, 24 cases), **L2** ($5 \leq C \leq 7$, 45 cases), **L3** ($C \geq 8$, 46 cases).

5.3 Evaluation Metrics

We adopt the metric suite of Kavathekar et al. (2025):

- **FSP** (Function Selection Precision): Binary indicator of correct tool prediction.
- **ACS** (Argument Coverage Score): F_1 on argument *name* overlap.
- **AVC** (Argument Value Correctness): Fraction of correctly named arguments whose values also match.
- **Task Accuracy**: F_1 on the combined set $\{\text{tool}\} \cup \{k=v\}$, jointly capturing tool selection and parameter correctness.
- **Correct Ratio (CR)**: Strict binary indicator—1.0 only when Task Acc = 1.0, i.e. the entire function call is perfectly correct.

Model	Family	Params	RAM
Qwen3.5-0.8B	Qwen3.5 (2025)	0.8B	1.3 GB
Qwen3.5-2B	Qwen3.5 (2025)	2B	2.2 GB
MiniCPM5-1B	MiniCPM (2024)	1B	1.5 GB
EXAONE-4.0-1.2B	EXAONE-4.0 (2025)	1.2B	2.0 GB
Gemma-3-1B-it	Gemma-3 (2025)	1B	1.6 GB
LFM2.5-1.2B	LFM2 (2025)	1.2B	1.5 GB

Table 3: Models evaluated. All use Q8_0 GGUF quantization via llama.cpp. RAM is average peak RSS.

7 Results

7.1 Overall Performance

Table 4 presents the full results. Figure 1 shows the accuracy–latency Pareto frontier and the error taxonomy breakdown.

Key findings (RQ1, RQ2). (1) Sub-2B SLMs can reliably call financial tools on consumer hardware (**RQ1**: yes), with the Qwen3.5 family achieving >89% Task Accuracy in dual mode. This eliminates the need for cloud inference and keeps all user financial data strictly local. (2) While improved prompt engineering has substantially raised single-agent performance (Qwen3.5-2B reaches 92.0% Task Accuracy), dual-agent decomposition remains critical for two reasons (**RQ2**): (a) Strict correctness—Correct Ratio improves from 60.2% to 77.6% (+17.4 pp) for Qwen3.5-2B, meaning 17 more queries out of every 100 are *perfectly* correct with no parameter errors. For financial applications where a single wrong parameter can produce misleading analytics, this improvement is substantial. (b) Small model enablement—Qwen3.5-0.8B improves from 79.8% to 89.1% Task Accuracy (+9.3 pp) and *halves* its latency (5.8 s→3.0 s) in dual mode, making sub-1B models viable for memory-constrained devices. (3) MiniCPM5-1B emerges as a strong mid-tier performer, reaching 74.3% Task Accuracy in dual mode with 43.5% Correct Ratio—a notable result for a 1B model.

7.2 Per-Tool-Group Analysis

Table 5 shows per-group accuracy for the top two dual-mode models. Figure 2 shows the full model × category heatmap. `get_top_expenses` is nearly solved (98.9% for Qwen3.5-2B)—its parameter space is simple and unambiguous. `time_series` is the most challenging (74.4% for Qwen3.5-2B), owing to diverse date range specifications and relative time expressions (“past X months”) that require temporal reasoning.

7.3 Complexity Tier Analysis

Figure 3 presents the per-model accuracy breakdown across complexity tiers in dual mode. Qwen3.5-2B degrades gracefully from 96% (L1) to 82% (L3), while Qwen3.5-0.8B shows a similar trajectory (95%→84%). L3 queries involve constructs like cross-year range comparisons (“compare groceries nov 2024 to apr 2025 vs nov 2025 to apr 2026”) requiring up to 10 parameters with interdependent temporal semantics.

All metrics are averaged across 5 repetitions per test case (temperature = 0.1) with 95% bootstrap confidence intervals (Efron, 1979) ($n = 2000$ resamples).

5.4 Error Taxonomy

Each prediction is classified into one of seven mutually exclusive error types: CORRECT, TOOL_WRONG, PARSE_FAILURE, PARAM_MISSING, PARAM_VALUE_WRONG, PARAM_EXTRA, FORMAT_VIOLATION. This enables targeted analysis of *what* fails rather than just *how much*, which is essential for assessing the safety profile of on-device financial assistants.

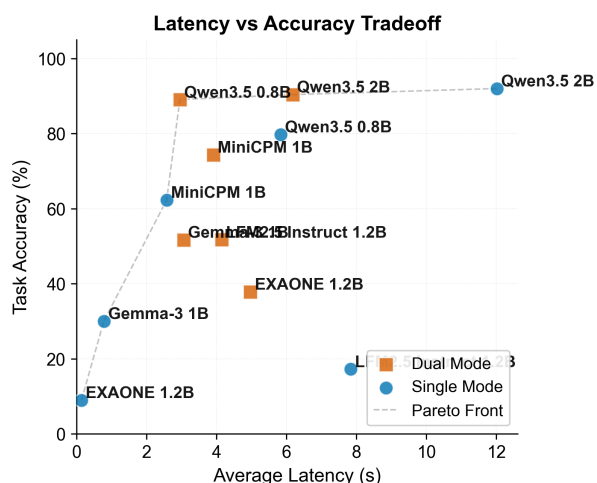
6 Experimental Setup

6.1 Models

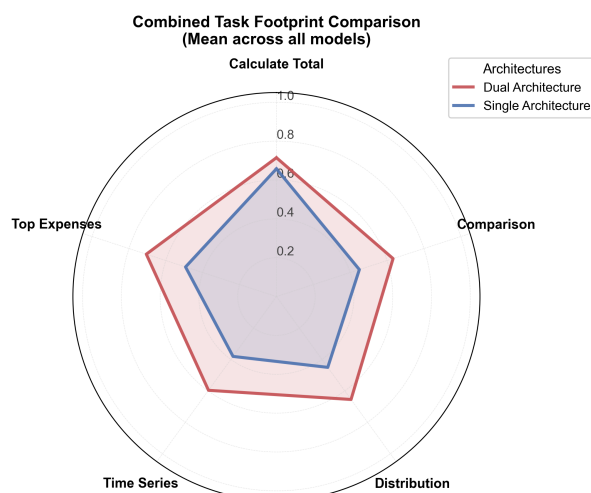
All six models (Table 3) use Q8_0 GGUF quantization and run via llama.cpp (Gerganov, 2023) with full GPU offload on Apple Metal.

6.2 Hardware and Inference Protocol

Experiments run on an Apple M1 (8-core CPU, 7-core GPU, 8 GB unified memory)—a consumer laptop representative of privacy-constrained deployment environments. Parameters: temperature = 0.1, min_p = 0.05, n_ctx = 4096, stop tokens [“\n\n”, “”]. Each model is warmed up through all six prompt templates (one router + five specialists) before evaluation begins. Each of the 115 test cases is run 5 times per model per architecture, yielding $115 \times 5 \times 6 \times 2 = 6,900$ total observations.



(a) Accuracy–latency Pareto frontier. The dual-mode Qwen3.5-0.8B (orange square, ≈ 3 s, 89%) sits on the efficient frontier, offering the best accuracy per unit latency. Single-mode Qwen3.5-2B reaches the highest Task Accuracy (92%) but at 4 $\times$ the latency (12 s).



(b) Task footprint (mean Task Accuracy across all models). The dual architecture (red) uniformly expands coverage over single (blue) on all five task groups, with the largest gains on time series and comparison tools.

Figure 1: System-level results. (a) Dual-agent Qwen3.5-0.8B dominates the efficiency frontier for privacy-preserving deployments. (b) Dual-agent decomposition improves coverage across all task categories.

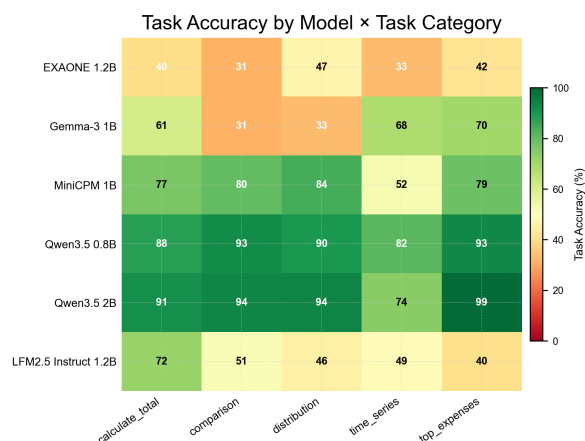


Figure 2: Task Accuracy (%) by Model $\times$ Tool Category in dual mode. Qwen3.5 models achieve $>74\%$ across all categories, while weaker models show pronounced gaps in comparison and time series tasks.

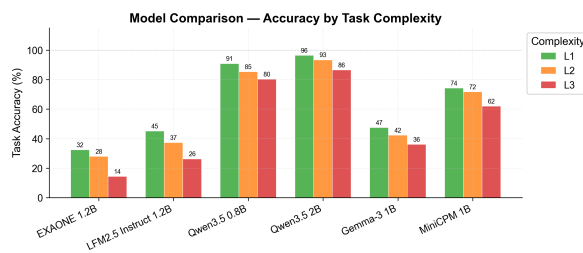


Figure 3: Task Accuracy by complexity tier (L1/L2/L3) per model in dual mode. The Qwen3.5 family degrades gracefully with increasing complexity.

7.4 Error Taxonomy Analysis

Figure 4 visualises the full error distribution. The shift in error profiles answers **RQ3** directly.

For Qwen3.5-2B: single-agent produces 22.6% `PARAM_EXTRA` and 7.3% `PARAM_MISSING`; dual-agent reduces `PARAM_EXTRA` to 1.9% while `PARAM_MISSING` remains at 8.0%. The elimination of `PARAM_EXTRA` errors is particularly important for financial safety: a spurious months=1 filter added to “how much did I spend on food?” silently restricts the query to the last month, producing a misleadingly low total. Dual-agent’s

residual `PARAM_MISSING` errors are a *safer* failure mode—the system omits a filter, producing a broader result, which is less likely to cause financial misjudgements.

For Qwen3.5-0.8B: single-agent produces 32.5% `PARAM_MISSING` and 11.8% `PARAM_EXTRA`; dual-agent reduces these to 12.9% and 11.1% respectively, shifting the model from a high-variance to a more predictable failure profile.

7.5 Token Efficiency

Figure 5 shows that despite dual-agent using $\approx 2,700$ prompt tokens per query (vs. $\approx 5,500$ for single-agent due to the comprehensive multi-tool prompt), it achieves equivalent or superior accuracy. The single-agent pipeline processes *more* context but converts it less efficiently into correct tool calls, because the model must simultaneously attend to five tool schemas and cross-tool disambiguation.

Mode	Model	FSP	ACS	AVC	Task Acc	CR	Latency (s)	RAM (GB)
Single	Qwen3.5-2B	96.5	—	97.6	92.0	60.2	12.0	2.2
	Qwen3.5-0.8B	95.0	—	91.2	79.8	40.2	5.8	1.3
	MiniCPM5-1B	71.1	—	86.7	62.3	13.0	2.6	1.5
	Gemma-3-1B-it	27.0	—	47.5	30.0	0.9	0.8	1.6
	LFM2.5-1.2B	28.7	—	11.7	17.2	3.1	7.8	1.5
	EXAONE-4.0-1.2B	19.3	—	0.3	8.9	0.3	0.1	2.0
Dual	Qwen3.5-2B	92.0	91.7	97.4	90.4	77.6	6.2	2.2
	Qwen3.5-0.8B	94.3	90.0	96.8	89.1	61.7	3.0	1.3
	MiniCPM5-1B	84.9	—	81.6	74.3	43.5	3.9	1.5
	LFM2.5-1.2B	69.6	—	57.2	51.8	16.3	4.1	1.5
	Gemma-3-1B-it	55.1	60.0	71.6	51.7	16.5	3.1	1.6
	EXAONE-4.0-1.2B	90.6	3.5	3.3	37.8	3.0	5.0	2.0

Table 4: Main results (validated metrics, %). **Bold** = best per metric. FSP = Function Selection Precision; ACS = Argument Coverage Score; AVC = Argument Value Correctness; Task Acc = F_1 -based task accuracy; CR = strict exact-match Correct Ratio. ACS is omitted (—) where the single-agent architecture’s joint output makes isolated argument coverage less meaningful. Latency and RAM are per-query averages.

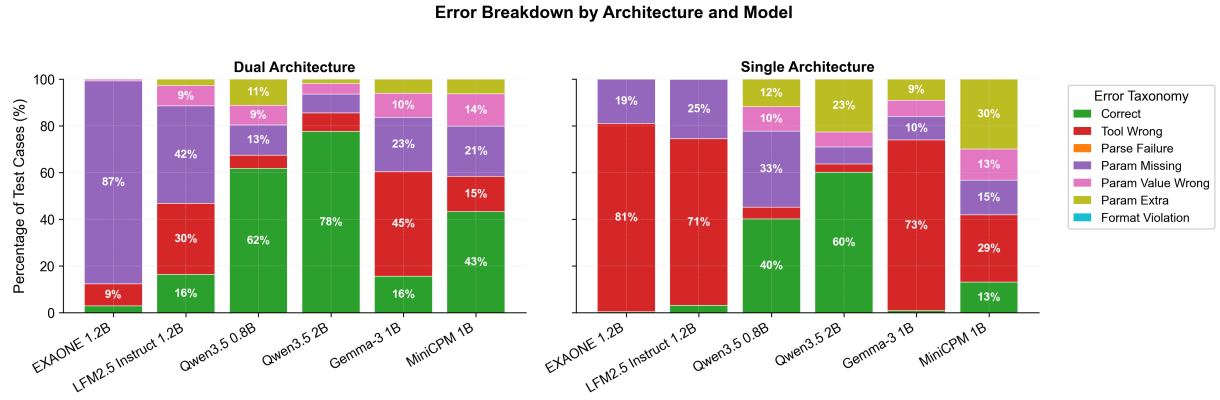


Figure 4: Error taxonomy breakdown (% of test cases) per model and architecture. Single-agent Qwen3.5-2B (right panel) produces 22.6% `PARAM_EXTRA` errors—spurious parameters that could lead to incorrect financial calculations. Dual-agent (left) eliminates most of these, consolidating residual errors into the more predictable `PARAM_MISSING` category.

Group	Qwen3.5-2B		Qwen3.5-0.8B	
	T.Acc	CR	T.Acc	CR
Time Series	74.4	60.9	81.6	54.8
Distribution	93.8	79.0	89.6	56.2
Comparison	93.7	73.6	92.5	71.4
Calc. Total	91.4	84.5	87.9	59.1
Top Expenses	98.9	92.4	93.3	64.8

Table 5: Per-group results in dual mode (validated, %). T.Acc = Task Accuracy, CR = Correct Ratio.

Model (Dual)	Δ Task Acc	Δ CR
MiniCPM5-1B	+1.17	+0.87
Qwen3.5-0.8B	+0.41	+1.57
Qwen3.5-2B	+0.29	+0.87
LFM2.5-1.2B	+0.50	+0.00
Gemma-3-1B-it	+0.12	+0.00
EXAONE-4.0	+0.00	+0.00

Table 6: Validation layer impact (Δ in percentage points, dual mode). Gains are modest but consistent for capable models.

451 The dual-agent architecture processes less context
452 per call but achieves higher accuracy-per-token by
453 focusing each inference pass on a single subtask.

459 failure mode and validation cannot recover miss-
460 ing parameters, the primary bottleneck is structural
461 extraction, not surface normalisation.

454 7.6 Validation Layer Impact

455 The validation layer (Table 6) contributes mod-
456 est gains, primarily correcting category cas-
457 ing/normalisation. Its limited impact is *informative*
458 since `PARAM_MISSING` is the dominant residual

462 8 Discussion

463 **Privacy through Task Decomposition.** The
464 dual-agent architecture serves a dual purpose: it im-
465 proves accuracy *and* makes privacy-preserving lo-

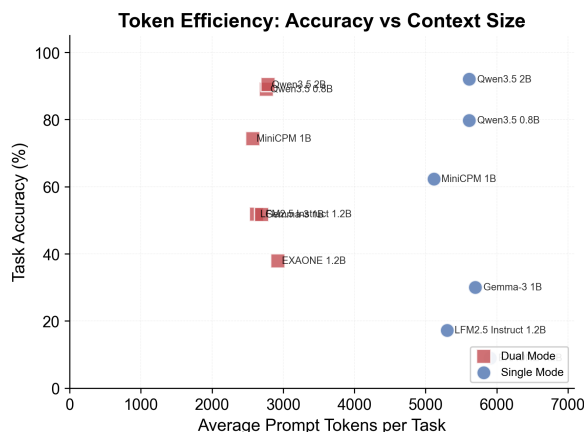


Figure 5: Token efficiency: Task Accuracy vs. average prompt tokens per query. Dual-agent configurations (red) achieve higher accuracy with fewer total tokens than their single-agent counterparts (blue), demonstrating superior context utilisation.

put, rather than a semantic failure. The function-
masking approach of Lin et al. (2024) or the
functional-token design of Chen and Li (2024) may
offer paths to resolving such failures via targeted
fine-tuning.

L3 complexity remains a hard frontier. Even
the best model achieves only 59.6% strict Correct
Ratio on L3 queries in dual mode, which involve
cross-year range comparisons requiring up to 10
interdependent parameters. Fine-tuning on domain-
specific examples using verified synthetic data (Liu
et al., 2024b) represents a promising avenue for
closing this gap while preserving the privacy bene-
fits of local inference.

**Deployment recommendations for privacy-
first systems.** For practitioners building privacy-
preserving financial assistants: (1) *Memory-
constrained* (≤ 2 GB available): Qwen3.5-0.8B
dual, 89.1% Task Acc, 3.0 s, 1.3 GB RAM. (2)
Accuracy-first: Qwen3.5-2B dual, 90.4% Task Acc,
77.6% CR, 6.2 s, 2.2 GB RAM. (3) *Budget hard-
ware*: MiniCPM5-1B dual, 74.3% Task Acc, 43.5%
CR, 3.9 s, 1.5 GB RAM. All configurations keep
user data entirely on-device.

Illustrative Case Study. Figure 6 shows a real
usage session of ExpenseSense. A user queried the
system in informal language—“can ya plot spend-
ing on snacks for past 14 months?”—which the
dual-agent Qwen3.5-0.8B pipeline correctly routed
to `plot_time_series` and extracted the pa-
rameters `{category: snacks, months: 14}`. The
resulting visualization revealed a clear declining
trend in weekly snack expenditure (from peaks
of ¥3,000–4,000 to under ¥1,000), which the
user reported influenced their subsequent spend-
ing behaviour. While anecdotal, this example il-
lustrates two properties of the system: (1) fully
private, on-device inference successfully handles
naturalistic queries over real personal financial
data, and (2) the resulting analytics are action-
able—precisely the value proposition that moti-
vates keeping financial AI local rather than cloud-
dependent.

9 Conclusion

We presented ExpenseSense, a human-centered
case study evaluating privacy-preserving on-device
financial tool calling with sub-2B quantized models.
Our central finding is that *architectural decomposi-
tion*—separating intent classification from param-
eter extraction into a router-specialist pipeline—

cal inference viable. By separating routing (single-
digit output) from extraction (focused JSON), dual-
agent reduces the structured output burden per call,
which is precisely the bottleneck that Kavathekar
et al. (2025) identify as the primary limitation
of SLMs for function calling. This cognitive of-
flooding is what makes sub-2B models competitive
with much larger cloud models for domain-specific
tasks, enabling fully private local execution.

**Why dual-agent still matters despite strong
single-agent prompts.** A natural question is
whether improved prompt engineering—which has
raised single-agent Qwen3.5-2B to 92.0% Task
Accuracy—has rendered the dual-agent architec-
ture unnecessary. Our results argue strongly
against this: (1) The 17.4 pp gap in Correct Ratio
(60.2%→77.6%) means that single-agent produces
wrong parameters in 40% of queries vs. 22% for
dual-agent. (2) For the privacy-optimal Qwen3.5-
0.8B (which fits in <1.5 GB RAM), dual-agent
provides a +9.3 pp Task Accuracy boost and halves
latency, because shorter, more focused prompts
generate faster on small models. (3) The error tax-
onomy shift from `PARAM_EXTRA` (dangerous in
finance) to `PARAM_MISSING` (safer) represents a
qualitative safety improvement.

The EXAONE anomaly. EXAONE-4.0-1.2B
achieves 90.6% FSP in dual mode—excellent
routing—but averages near-empty specialist out-
puts (ACS = 3.5%). This strongly suggests a chat-
template or instruction-following constraint in the
GGUF conversion that truncates structured out-

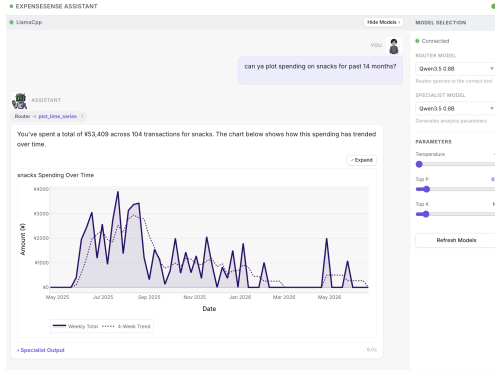


Figure 6: Real usage session: a user queries snack spending trends in informal language. The dual-agent Qwen3.5-0.8B pipeline correctly invokes `plot_time_series`, producing an actionable spending trend—entirely on-device with no data transmitted externally.

remains critical even as single-agent prompts improve: it unlocks strict correctness (77.6% Correct Ratio vs. 60.2%) and enables sub-1B models to reach 89.1% Task Accuracy at 3.0 s, a practical operating point where no user data ever leaves the device. Error taxonomy analysis reveals that dual agent decomposition not only improves accuracy but produces *safer* failure modes: residual errors shift from dangerous parameter additions to more benign parameter omissions, a qualitative property desirable for financial safety. Several directions warrant future investigation. First, *fine-tuning* the best-performing models on domain-specific function-calling data could address the residual L3 failures while preserving on-device deployment. Second, extending the benchmark to *multi-turn* interactions would capture realistic conversational patterns. Third, *multilingual* query support would broaden applicability across diverse user populations. Finally, evaluating lower-bit quantisation formats (Q4_K_M) could further reduce the memory footprint, making privacy-preserving financial AI accessible on even more constrained devices such as smartphones.

Limitations

- The benchmark targets a single expense tracking application; generalisability to other financial domains (trading, compliance, auditing) requires further investigation.
- All models use Q8_0 quantisation; lower-bit formats (Q4_K_M) may yield different accuracy–efficiency tradeoffs.

- Queries are in English and reflect a Japan-based personal finance context (categories such as “combinis”, “nomikais”, “shinkansen”). Multilingual and cross-cultural diversity is not covered.
- Ground-truth labels are deterministic; some queries may admit legitimate alternative tool selections.
- Test cases were authored by a single developer; crowdsourcing or user study collection would improve query diversity.
- We do not evaluate the *downstream correctness* of the financial analytics produced—only whether the tool call is correctly formed. Production deployment would require additional safeguards against calculation errors.
- Our privacy analysis is architectural (no data leaves the device) rather than cryptographic; we do not address model inversion or side-channel attacks on the local device itself.

Ethics Statement

ExpenseSense is designed for on-device inference specifically to preserve user financial privacy. No real user financial data was used in the benchmark; all test cases reference synthetic expense categories. The illustrative case study (Figure 6) uses the first author’s own data, shared with informed consent; no other user data was collected. The system is intended to complement, not replace, professional financial advice. We recognise that even on-device models may produce incorrect financial summaries, and advocate for clear user-facing disclaimers in production deployments.

Data and Code Availability

Code, prompts, benchmark data, and full results will be released on GitHub upon acceptance at [anonymised for review]. Released artefacts include: all 115 test cases with ground-truth labels, the complete router and specialist prompt templates, inference configuration details, raw and validated CSV result files, and the benchmark runner, plotting scripts, and the exact model GGUFs used in this study to ensure complete reproducibility.

623 References

- 624 Marah Abdin, Jyoti Aneja, Hany Awadalla, and 1 others.
625 2024. Phi-3 technical report: A highly capable lan-
626 guage model locally on your phone. *arXiv preprint*
627 *arXiv:2404.14219*.
- 628 Yupeng Cao, Haohang Li, Weijin Liu, Wenbo Cao,
629 Anke Xu, Lingfei Qian, and 1 others. 2026. Fin-
630 Trace: Holistic trajectory-level evaluation of LLM
631 tool calling for long-horizon financial tasks. *arXiv*
632 *preprint arXiv:2604.10015*.
- 633 Wei Chen and Zhiyuan Li. 2024. Octopus v2: On-
634 device language model for super agent. *arXiv*
635 *preprint arXiv:2404.01744*.
- 636 Zhiyu Chen, Wenhui Chen, Charese Smiley, Sameena
637 Shah, Iana Borova, Dylan Langdon, Reema Moussa,
638 Matt Beane, Ting-Hao Huang, Bryan Routledge, and
639 William Yang Wang. 2021. *FinQA: A dataset of*
640 *numerical reasoning over financial data*. In *Proceed-*
641 *ings of the 2021 Conference on Empirical Methods*
642 *in Natural Language Processing*, pages 3697–3711.
- 643 B. Efron. 1979. Bootstrap methods: Another look at the
644 jackknife. *The Annals of Statistics*, 7(1):1–26.
- 645 Gemma Team. 2025. Gemma 3 technical report. *arXiv*
646 *preprint arXiv:2503.19786*.
- 647 Georgi Gerganov. 2023. *llama.cpp*.
- 648 Tom Gunter, Zirui Wang, Chong Wang, and 1 others
649 2024. Apple intelligence foundation language mod-
650 els. *arXiv preprint arXiv:2407.21075*.
- 651 Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu
652 Cui, Xiang Long, and 1 others. 2024. MiniCPM
653 Unveiling the potential of small language models
654 with scalable training strategies. *arXiv preprint*
655 *arXiv:2404.06395*.
- 656 Ishan Kavatkar, Raghav Donakanti, Ponnurangam Ku-
657 maraguru, and Karthik Vaidhyanathan. 2025. *Small*
658 *models, big tasks: An exploratory empirical study*
659 *on small language models for function calling*. In
660 *Proceedings of the 29th International Conference on*
661 *Evaluation and Assessment in Software Engineering*,
662 pages 1117–1126.
- 663 LG AI Research. 2025. EXAONE 4.0: Unified large
664 language models integrating non-reasoning and rea-
665 soning modes. *arXiv preprint arXiv:2507.11407*.
- 666 Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li,
667 Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing
668 Xu, Xiang Wang, Yi Sun, and 1 others. 2024. Per-
669 sonal LLM agents: Insights and survey about the
670 capability, efficiency and security. *arXiv preprint*
671 *arXiv:2401.05459*.
- 672 Qiqiang Lin, Muning Wen, Qiuying Peng, Guanyu
673 Nie, Junwei Liao, Jun Wang, and 1 others. 2024
674 Hammer: Robust function-calling for on-device lan-
675 guage models via function masking. *arXiv preprint*
676 *arXiv:2410.04587*.
- 677 Liquid AI. 2025. Lfm2 technical report. *arXiv preprint*
678 *arXiv:2511.23404*.
- 679 Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen
680 Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong,
681 Ernie Chang, Yangyang Shi, Raghuraman Krish-
682 namoorthi, Liangzhen Lai, and Vikas Chandra. 2024a.
683 *MobileLLM: Optimizing sub-billion parameter lan-*
684 *guage models for on-device use cases*. In *Proceed-*
685 *ings of the 41st International Conference on Machine*
686 *Learning*, pages 32431–32454.
- 687 Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu,
688 Tian Lan, and 1 others. 2024b. APIGen: Auto-
689 mated pipeline for generating verifiable and diverse
690 function-calling datasets. *Advances in Neural Inform-*
691 *ation Processing Systems*, 37.
- 692 Helen Nissenbaum. 2004. Privacy as contextual in-
693 tegrity. *Washington Law Review*, 79(1):119–158.
- 694 Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Char-
695 lie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and
696 Joseph E. Gonzalez. 2025. *The Berkeley function*
697 *calling leaderboard (BFCL): From tool use to agentic*
698 *evaluation of large language models*. In *Proceedings*
699 *of the 42nd International Conference on Machine*
700 *Learning*, pages 48371–48392.
- 701 Shishir G. Patil, Tianjun Zhang, Xin Wang, and
702 Joseph E. Gonzalez. 2024. *Gorilla: Large language*
703 *model connected with massive APIs*. In *Advances in*
704 *Neural Information Processing Systems*, volume 37.
- 705 Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan
706 Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang,
707 Bill Qian, and 1 others. 2024. *ToolLLM: Facilitating*
708 *large language models to master 16000+ real-world*
709 *APIs*. In *Proceedings of the 12th International Con-*
710 *ference on Learning Representations*.
- 711 Qwen Team. 2025. Qwen3 technical report. *arXiv*
712 *preprint arXiv:2505.09388*.
- 713 Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta
714 Raileanu, Maria Lomeli, Eric Hambro, Luke Zettle-
715 moyer, Nicola Cancedda, and Thomas Scialom. 2023.
716 Toolformer: Language models can teach themselves
717 to use tools. In *Advances in Neural Information Pro-*
718 *cessing Systems*, volume 36.
- 719 Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li,
720 Weiming Lu, and Yueting Zhuang. 2023. Hugging-
721 GPT: Solving AI tasks with ChatGPT and its friends
722 in Hugging Face. In *Advances in Neural Information*
723 *Processing Systems*, volume 36.
- 724 Yewei Song, Xunzhu Tang, Cedric Lothritz, Saad Ezzini,
725 Jacques Klein, Tegawendé F. Bissyandé, Andrey
726 Boytsov, Ulrick Ble, and Anne Goujon. 2025. *Call-*
727 *Navi: A challenge and empirical study on LLM func-*
728 *tion calling and routing*. In *Proceedings of the 29th*
729 *International Conference on Evaluation and Assess-*
730 *ment in Software Engineering*.

- 731 Robin Staab, Mark Vero, Mislav Balunović, and Mar-
 732 tin Vechev. 2024. Beyond memorization: Violating
 733 privacy via inference with large language models. In
 734 *Proceedings of the 12th International Conference on*
 735 *Learning Representations*.
- 736 Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han,
 737 Qiao Liang, and Le Sun. 2023. ToolAlpaca: Gener-
 738 alized tool learning for language models with 3000
 739 simulated cases. *arXiv preprint arXiv:2306.05301*.
- 740 Jiajun Xu, Zhiyuan Li, Wei Chen, Qun Wang, Xin Gao,
 741 Qi Cai, and Ziyuan Ling. 2024. On-device language
 742 models: A comprehensive review. *arXiv preprint*
 743 *arXiv:2409.00088*.
- 744 Hongyang Yang, Xiao-Yang Liu, and Christina Dan
 745 Wang. 2023. [FinGPT: Open-source financial large](#)
 746 [language models](#). In *International Symposium on*
 747 *Large Language Models for Financial Services (Fin-*
 748 *LLM 2023) at IJCAI 2023*.
- 749 Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo
 750 Sun, and Yue Zhang. 2024. A survey on large lan-
 751 guage model (LLM) security and privacy: The good,
 752 the bad, and the ugly. *High-Confidence Computing*,
 753 4(2):100211.
- 754 Jie Zhu, Yimin Tian, Boyang Li, Kehao Wu, Zhongzhi
 755 Liang, Junhui Li, Xianyin Zhang, Lifan Guo, Feng
 756 Chen, Yong Liu, and Chi Zhang. 2026. FinMCP-
 757 Bench: Benchmarking LLM agents for real-world
 758 financial tool use under the model context protocol.
 759 *arXiv preprint arXiv:2603.24943*.

760 A Complexity Scoring Features

Feature	Range	Description
n_p	0+	Number of expected parameters
d_{date}	0–4	Date complexity: none / year / month / range / specific day
d_{cat}	0–2	Category distance: exact / alias / semantic
d_{rel}	0–1	Relative time expression (“past X months”)
d_{multi}	0–2	Compound parameter groups
d_{abbr}	0–1	Abbreviated year formats (“’24”)
d_{hol}	0–2	Holiday/calendar knowledge required

Table 7: Seven features of the complexity scoring function (Eq. 1).

761 B Sample Test Cases

ID	Query	Tool	Expected Params	Tier
TS01	Can ya show spending on food for past 6 months?	plot_time_series	category=Food, months=6	L2
DI06	show breakdown of expenses for june 21 2024	plot_distribution	year=2024, month=6, day=21	L2
CP24	compare groceries nov 2024 to apr 2025 vs nov 2025 to apr 2026	plot_comparison_bars	category=grocery, y1=2024, sm1=11, ey1=2025, em1=4, y2=2025, sm2=11, ey2=2026, em2=4	L3
CT07	sum total spent on metro for july 2024?	calculate_total	category=tokyo metro, year=2024, month=7	L2
TP06	top 5 expenses past month? disregard rent	get_top_expenses	n=5, months=1, ignore_rent=true	L2

Table 8: Sample test cases. Note informal language, abbreviations, and semantic mappings (“metro”→“tokyo metro”, “groceries”→“grocery”).

762 C Qualitative Error Examples

763 Table 9 illustrates representative failures for the dominant error types in Qwen3.5-2B (dual mode).

Error Type	Query	Expected	Model Output
PARAM_MISSING	“Show spending trend for the last 3 months”	{months: 3}	{ } — months omitted in the relative-time case
PARAM_VALUE_WRONG	“compare groceries nov 2024 to apr 2025 vs nov 2025 to apr 2026”	{em1: 4, em2: 4}	{em1: 11, em2: 11} — confuses start month for end month in cross-year range
PARAM_EXTRA	“How much did I spend on food?”	{category: Food}	{category: Food, months: 1} — spurious time filter restricts scope

Table 9: Representative failure cases for Qwen3.5-2B (dual mode). Note that PARAM_EXTRA is particularly dangerous in financial contexts: the spurious months=1 filter silently restricts “total food spending” to “last month’s food spending”.